

LoRA vs ControlNet: Conditioning in Diffusion Models

A modular experimental framework to study conditioning mechanisms in diffusion models, comparing **ControlNet**, **LoRA-based conditioning**, and **input concatenation** baselines on MNIST.

🔗 Research Hypothesis

Conditioning signals are low-rank relative to the denoiser, and therefore LoRA adapters are sufficient to achieve comparable performance to full ControlNet with significantly fewer parameters.

📁 Project Structure

```
PYPY/  
├── model.py           # All model architectures (UNet, ControlNet, LoRA)  
├── dataset.py        # Dataset classes and conditioning generators  
├── train.py          # Training loop and experiment runners  
├── evaluate.py       # Evaluation metrics and model assessment  
├── visualize_results.py # Visualization utilities  
├── checkpoints/      # Saved model weights  
└── visualizations/   # Generated plots and samples
```

🏗️ Model Architectures

1. Base UNet (Denoiser)

The backbone is a **small UNet-based DDPM** trained from scratch on MNIST (no pretrained weights).

```
Architecture:  
├── Input: (B, 1, 28, 28) noisy image + timestep t  
├── Time Embedding: Sinusoidal → MLP → 256-dim  
├── Encoder:  
│   ├── conv_in: 1 → 64 channels  
│   ├── down1: 2× ResBlock (64 → 64) + Downsample → 14×14  
│   └── down2: 2× ResBlock (64 → 128) + Downsample → 7×7  
├── Middle:  
│   ├── ResBlock (128 → 128)  
│   ├── Self-Attention (128 channels)  
│   └── ResBlock (128 → 128)  
├── Decoder:  
│   ├── up2: Upsample + Skip + 2× ResBlock (256 → 64) → 14×14  
│   └── up1: Upsample + Skip + 2× ResBlock (128 → 64) → 28×28
```

```
|
└─ Output: conv_out → (B, 1, 28, 28) predicted noise
```

Key Components:

- **ResBlock:** Conv → GroupNorm → SiLU → Conv + Time embedding injection + Skip connection
- **Attention:** Self-attention with Q, K, V projections
- **Normalization:** GroupNorm (8 groups)
- **Activation:** SiLU throughout

Parameters: ~1.2M total

2. ControlNet (Full Copy Baseline)

Traditional ControlNet architecture: a **complete copy of the UNet encoder** that processes the conditioning signal and injects residuals via zero convolutions.

```
ControlNet Architecture:
├─ Input: (B, C, 28, 28) conditioning signal + timestep t
├─ Condition Encoder (copy of UNet encoder):
│   ├── conv_in: C → 64
│   ├── down1: 2× ResBlock + Downsample
│   ├── down2: 2× ResBlock + Downsample
│   └── mid: 2× ResBlock + Attention
├─ Zero Convolutions (5 injection points):
│   ├── After down1 blocks → inject to UNet down1
│   ├── After downsample1 → inject to UNet
│   ├── After down2 blocks → inject to UNet down2
│   ├── After downsample2 → inject to UNet
│   └── After mid block → inject to UNet mid
└─ Output: List of 5 control tensors added to base UNet features
```

Parameters: ~1.8M (base UNet) + ~0.9M (ControlNet) = **~2.7M total**

3. LoRA-ControlNet (Parameter-Efficient)

Our proposed method: a **lightweight condition encoder** combined with **LoRA adapters** that modulate UNet features.

```
LoRA-ControlNet Architecture:
├─ Lightweight Condition Encoder:
│   ├── Conv 32 (3×3) → SiLU
│   ├── Conv 64 (3×3, stride=2) → SiLU # 28→14
│   └── Conv 128 (3×3, stride=2) → SiLU # 14→7
```

```

├── LoRA Adapters (configurable injection):
│   ├── Early: LoRA at down1 (rank r)
│   ├── Mid: LoRA at middle block (rank r)
│   └── All: LoRA at early + down + mid
├── LoRA Layer Structure:
│   ├── lora_A: (in_channels → rank) via 1×1 conv
│   ├── lora_B: (rank → out_channels) via 1×1 conv
│   └── Output: lora_B(lora_A(x)) * (α/r)

```

LoRA Parameters (rank=4, mid-only):

- Condition encoder: ~50K
- LoRA adapters: ~4K
- **Total: ~1.25M** (vs 2.7M for ControlNet = **54% reduction**)

4. Vector LoRA-ControlNet

For **scalar/vector conditioning** (stroke thickness, center/scale), we use MLPs to project to spatial features:

```

Vector Conditioning Pipeline:
├── Input: (B, D) vector (D=1 for thickness, D=3 for center/scale)
├── Vector Encoder MLP:
│   ├── Linear(D → 256) → SiLU
│   └── Linear(256 → 512) → SiLU
├── Spatial Projection:
│   └── Linear(512 → C × H × W) → Reshape to (B, C, H, W)
├── LoRA Modulation:
│   └── Apply LoRA adapters to projected features

```

5. Input Concatenation Baseline

Simple baseline: concatenate conditioning to input channels.

```

ConcatConditionUNet:
├── Input: cat([noisy_image, condition], dim=1) → (B, 1+C, 28, 28)
└── UNet with in_channels = 1 + conditioning_channels

```

Conditioning Types

Image-based Conditioning (B, C, H, W)

Type	Channels	Description
edge	1	Canny edge detection
sobel	1	Sobel gradient magnitude
skeleton	1	Morphological skeleton
inpainting	2	Masked image + binary mask

Vector-based Conditioning (B, D)

Type	Dimensions	Description
stroke_thickness	1	Distance transform estimate
center_scale	3	(cx, cy, scale) normalized
color_histogram	16/48	Grayscale/RGB histogram

 Inpainting Mask Types

Traditional Masks

Mask Type	Description
top_half	Mask top 50% of image
bottom_half	Mask bottom 50% of image
left_half / right_half	Mask left/right halves
center	Mask center 50% region
random_rect	Random rectangle

Smart Digit-Aware Masks (NEW)

Mask Type	Description
digit_percentage	Randomly mask X% of digit pixels only
digit_contiguous	Mask X% of digit in contiguous region
digit_top	Mask top X% of digit pixels
digit_bottom	Mask bottom X% of digit pixels

The `--mask_percentage` parameter controls how much of the digit is hidden (0-100%).

 Experiments

1. Rank Ablation

Tests LoRA ranks: 1, 2, 4, 8, 16

```
python train.py --experiment rank_ablation --conditioning inpainting --epochs 5
```

2. Layer Ablation

Tests injection points: early, mid, all

```
python train.py --experiment layer_ablation --conditioning inpainting --epochs 5
```

3. Mask Percentage Ablation

Tests masking: 20%, 40%, 60%, 80% of digit

```
python train.py --experiment mask_percentage_ablation --epochs 5
```

4. Full Model Comparison

Compares: ControlNet vs Concat vs LoRA (mid) vs LoRA (all)

```
python train.py --experiment full_comparison --conditioning inpainting --epochs 5
```

5. Conditioning Type Ablation

Tests: edge, sobel, inpainting

```
python train.py --experiment conditioning_ablation --epochs 5
```

Quick Start

Training

```
# List available datasets and conditioning types
python train.py --list_datasets

# Train LoRA with smart digit masking (50% hidden)
python train.py --model_type lora --conditioning inpainting \
    --mask_type digit_percentage --mask_percentage 50 --epochs 10

# Train ControlNet baseline
```

```
python train.py --model_type controlnet --conditioning inpainting --epochs 10

# Train with contiguous mask (more realistic occlusion)
python train.py --model_type lora --conditioning inpainting \
  --mask_type digit_contiguous --mask_percentage 40 --epochs 10
```

Evaluation

```
python evaluate.py
```

 CLI Reference

```
usage: train.py [-h] [--model_type {baseline,concat,controlnet,lora}]
               [--rank RANK] [--inject_layer {early,mid,all}]
               [--dataset {mnist,svhn,clevr}]
               [--conditioning {edge,sobel,skeleton,inpainting,...}]
               [--mask_type {digit_percentage,digit_contiguous,...}]
               [--mask_percentage MASK_PERCENTAGE]
               [--epochs EPOCHS] [--batch_size BATCH_SIZE]
               [--lr LR] [--timesteps TIMESTEPS]
               [--experiment {rank_ablation,layer_ablation,...}]
               [--list_datasets]

Arguments:
  --model_type      Model architecture (default: lora)
  --rank            LoRA rank (default: 4)
  --inject_layer    Where to inject LoRA (default: mid)
  --conditioning    Conditioning signal type (default: edge)
  --mask_type       Mask strategy for inpainting (default: digit_percentage)
  --mask_percentage Percentage of digit to mask (default: 50)
  --epochs          Training epochs (default: 10)
  --batch_size      Batch size (default: 128)
  --lr             Learning rate (default: 1e-4)
  --timesteps       Diffusion timesteps (default: 1000)
  --experiment      Run predefined ablation study
```

 Evaluation Metrics

Metric	Description
MSE	Mean squared error between generated and target
Edge Alignment	Overlap between generated edges and condition edges
Structural Similarity	Normalized cross-correlation

 Technical Details

Diffusion Process

- **Forward:** $x_t = \sqrt{\bar{\alpha}_t} * x_0 + \sqrt{(1-\bar{\alpha}_t)} * \epsilon$
- **Objective:** Predict noise ϵ (DDPM)
- **Schedule:** Linear β from 1e-4 to 0.02 over 1000 steps
- **Sampling:** DDPM reverse process with variance

Training

- **Optimizer:** AdamW (weight_decay=0.01)
- **LR Schedule:** Cosine annealing
- **Gradient Clipping:** Max norm 1.0
- **Batch Size:** 128 (default)

LoRA Scaling

- $\alpha = r$ (scaling factor equals rank)
- **Output:** $\Delta W = B @ A * (\alpha/r)$

 Dependencies

```
torch>=1.9.0
torchvision
numpy
opencv-python
scipy
matplotlib
scikit-learn
tqdm
```

 Supported Datasets

Dataset	Size	Channels	Description
MNIST	28×28	1	Handwritten digits
SVHN	32×32	3	Street view house numbers
CLEVR	64×64	3	Synthetic 3D shapes

 Citation

If you use this code for research, please cite:

```
@misc{lora-controlnet-mnist,  
  title={LoRA vs ControlNet: Parameter-Efficient Conditioning in Diffusion  
Models},  
  year={2026},  
  note={Experimental framework for studying conditioning mechanisms}  
}
```

Key Findings (Expected)

1. **Parameter Efficiency:** LoRA achieves ~54% parameter reduction vs ControlNet
2. **Low-Rank Hypothesis:** Ranks 4-8 are sufficient for MNIST conditioning
3. **Injection Strategy:** Mid-block injection often matches full injection
4. **Mask Difficulty:** Performance degrades gracefully with increased masking percentage

Extending the Framework

Adding New Conditioning Types

1. Add generator in `dataset.py`:

```
@staticmethod  
def get_my_conditioning(img_np):  
    # Process image  
    return conditioning_output
```

2. Register in `_generate_conditioning()` method
3. Add to `get_conditioning_channels()` function

Adding New Datasets

1. Create dataset class inheriting `BaseConditioningDataset`
2. Register in `DATASET_CONFIG` and `get_dataset_class()`

License

MIT License